

JAVA

Herança & Classes

OOP PARTE 1

Uma apostila para entender, na prática, como o Java constrói objetos a partir de classes e como uma classe pode herdar comportamento de outra — usando um sistema de gestão de frota de veículos como fio condutor dos exemplos.

- 01 Classes, Objetos e Referências
- 02 Construtores e Sobrecarga
- 03 this e super — Acesso e Encadeamento
- 04 Herança com extends
- 05 static, POJO e a classe Object
- 06 Overloading vs Overriding

JavaStudiesHub

Classes, Objetos e Referências

O ponto de partida da orientação a objetos: o que é uma classe, o que é um objeto e como o Java lida com valores ausentes

O que é uma classe, afinal?

Toda a orientação a objetos em Java começa com uma distinção simples, mas que costuma confundir quem está chegando agora: uma **classe** não é uma coisa que existe na memória — ela é a descrição de uma coisa. É o molde. Pense em um estúdio de design de carros: antes de qualquer veículo sair de fábrica, existe um projeto técnico definindo quais peças ele tem e como cada uma se comporta. Esse projeto é a classe. Cada veículo que efetivamente sai da linha de produção — com chassi, motor e placa próprios — é um **objeto**, também chamado de **instância**.

Um objeto só passa a existir quando alguém usa a palavra-chave **new**: é nesse momento que o Java reserva um espaço na memória heap e cria uma cópia independente dos campos definidos na classe. A classe em si nunca ocupa espaço no heap — ela vive apenas como definição, carregada uma única vez pela JVM.

Variáveis guardam referências, não objetos

Um detalhe que gera bastante confusão no início: quando você atribui um objeto a uma variável, essa variável não guarda o objeto — ela guarda um **endereço** que aponta para onde o objeto mora no heap. Isso é chamado de referência. Consequência direta: se duas variáveis apontarem para o mesmo objeto, alterar o objeto através de uma delas afeta o que a outra enxerga, porque as duas apontam para o mesmo lugar — não existem duas cópias, existe um objeto com dois nomes.

```
Veiculo furgaoPrata = new Veiculo("Furgao", "PRT-0A12");

// Atribuir o objeto a outra variável NÃO cria uma cópia:
Veiculo veiculoDaManutencao = furgaoPrata; // mesma referência!

veiculoDaManutencao.registrarQuilometragem(500);
// furgaoPrata.getQuilometragem() também retorna 500,
// pois as duas variáveis apontam para o MESMO objeto no heap.
```

ANALOGIA

Pense na classe como a planta baixa de um modelo de veículo e no objeto como a unidade física que sai da fábrica seguindo essa planta. A mesma planta pode gerar centenas de veículos diferentes, cada um com seus próprios dados.

null e os valores padrão dos campos

null é o valor que representa "nenhum objeto". Toda variável de referência que ainda não recebeu um objeto atribuído aponta para null — ela existe, tem um tipo declarado, mas não aponta para nada no heap. O erro mais comum de quem está começando é tentar chamar um método em uma referência null: isso interrompe

o programa com um `NullPointerException` em tempo de execução, já que não existe objeto nenhum para receber aquela chamada.

Quando um objeto é criado, o Java preenche automaticamente os campos que não foram inicializados explicitamente, seguindo um valor padrão por tipo: os inteiros (`byte`, `short`, `int`, `long`, `char`) recebem 0, os tipos de ponto flutuante (`float`, `double`) recebem 0.0, o `boolean` recebe `false`, e qualquer tipo de referência — incluindo `String` — recebe `null`. Essa regra vale apenas para campos de instância; variáveis locais dentro de métodos não recebem valor padrão nenhum e precisam ser inicializadas manualmente antes de serem lidas.

```
public class Motorista {
    String nome; // null por padrão
    int anosDeExperiencia; // 0 por padrão
    boolean cnhValida; // false por padrão
    double avaliacaoMedia; // 0.0 por padrão
}
```

CUIDADO

Sempre que uma referência puder estar vazia, verifique antes de usá-la: `if (motorista != null) { motorista.exibirDados(); }`. Esse pequeno hábito evita boa parte dos erros em tempo de execução.

Encapsulamento: getters e setters

O encapsulamento estabelece que os campos de uma classe devem ser **private** — inacessíveis diretamente por quem está fora da classe. Todo acesso passa por métodos públicos: os **getters**, que devolvem o valor de um campo, e os **setters**, que o alteram. A grande vantagem não é burocracia — é controle: o setter pode recusar um valor inválido antes de gravá-lo, e a forma como o dado é armazenado internamente pode mudar sem que ninguém que usa a classe precise saber.

A convenção de nomes é previsível: um getter para o campo `placa` se chama `getPlaca()`; para campos `boolean`, usa-se o prefixo `is` (`isDisponivel()`). O setter correspondente é `setPlaca(String placa)`. Dentro do setter, a palavra-chave **this** resolve a ambiguidade quando o parâmetro tem o mesmo nome do campo que ele está atualizando.

```
public class Veiculo {
    private String placa;
    private int quilometragem;

    public String getPlaca() { return placa; }
    public void setPlaca(String placa) { this.placa = placa; }

    public int getQuilometragem() { return quilometragem; }
    public void setQuilometragem(int quilometragem) {
        if (quilometragem >= this.kilometragem) { // nunca anda pra trás
            this.kilometragem = quilometragem;
        }
    }
}
```

Construtores e Sobrecarga

Como o Java garante que todo objeto nasça em um estado válido

O papel do construtor

Um construtor é o bloco de código executado automaticamente sempre que um objeto é criado com `new`. Sua função é colocar o objeto em um estado inicial válido — nada mais. Ele compartilha o nome da classe e, diferente de um método comum, não declara nenhum tipo de retorno, nem mesmo `void`.

Quando nenhuma classe declara construtor algum, o compilador Java gera silenciosamente um construtor padrão sem parâmetros. Mas essa geração automática só acontece se você não declarar nenhum construtor manualmente: assim que um construtor qualquer é escrito, o construtor padrão deixa de ser gerado, e se ainda for necessário um construtor sem argumentos, ele precisa ser escrito explicitamente.

Sobrecarga de construtores

Sobrecarregar um construtor significa oferecer várias versões dele na mesma classe, cada uma recebendo uma lista diferente de parâmetros. O compilador decide qual construtor chamar olhando para os argumentos passados no momento da criação do objeto. Isso permite criar objetos com diferentes graus de detalhamento, sem forçar quem usa a classe a preencher tudo sempre.

```
public class Veiculo {
    private String modelo;
    private int anoFabricacao;

    // Construtor com poucos dados – usa valores padrão para o resto
    public Veiculo(String modelo) {
        this.modelo = modelo;
    }

    // Construtor completo
    public Veiculo(String modelo, int anoFabricacao) {
        this.modelo = modelo;
        this.anoFabricacao = anoFabricacao;
    }
}

// Duas formas de criar o mesmo tipo de objeto:
Veiculo generico = new Veiculo("Utilitario");
Veiculo detalhado = new Veiculo("Utilitario", 2022);
```

Constructor chaining: this() e super()

Constructor chaining é a técnica de fazer um construtor delegar parte do trabalho de inicialização para outro construtor, evitando repetir a mesma lógica em vários lugares. Existem duas formas: **this(argumentos)** chama outro construtor da própria classe, e **super(argumentos)** chama um construtor da superclasse. As duas só podem aparecer como a primeira instrução do construtor — o compilador rejeita qualquer código antes delas — e são mutuamente exclusivas: um mesmo construtor nunca pode ter **this()** e **super()** ao mesmo tempo.

```
public class VeiculoEletrico extends Veiculo {
    private int autonomiaKm;

    public VeiculoEletrico() {
        this("Modelo Base", 300); // delega para o construtor abaixo
    }

    public VeiculoEletrico(String modelo, int autonomiaKm) {
        super(modelo); // inicializa a parte herdada de Veiculo primeiro
        this.autonomiaKm = autonomiaKm;
    }
}
```

REGRA

O construtor mais completo é quem realmente inicializa todos os campos; os demais apenas delegam a ele através de **this()**. Quando existe herança, a classe pai é sempre inicializada primeiro, através de **super()**.

this e super fora dos construtores

this e **super** também servem para acessar campos e métodos — não só para encadear construtores. **this.campo** refere-se explicitamente ao campo da instância atual, o que é indispensável quando um parâmetro tem o mesmo nome do campo. **super.metodo()** chama a implementação do método definida na superclasse — muito útil quando uma subclasse quer complementar o comportamento do pai em vez de substituí-lo por completo. Nenhuma das duas palavras-chave pode ser usada dentro de um contexto **static**, já que ali não existe nenhuma instância associada.

```
public class Veiculo {
    protected String placa;
    public void exibirAlerta() {
        System.out.println("Alerta padrao de veiculo.");
    }
}

public class Caminhao extends Veiculo {
    @Override
    public void exibirAlerta() {
        super.exibirAlerta(); // reaproveita o comportamento do pai
        System.out.println("Alerta sonoro de re, veiculo pesado.");
    }

    public void setPlaca(String placa) {
        this.placa = placa; // this resolve a ambiguidade de nome
    }
}
```

Herança com extends

IS-A, visibilidade de campos herdados e sobrescrita de métodos

O mecanismo de herança

Herança é o recurso que permite que uma classe (a subclasse) adquira automaticamente todos os campos e métodos não-privados de outra classe (a superclasse), evitando reescrever código que já existe. A palavra-chave **extends** declara esse relacionamento, que semanticamente segue a regra **IS-A**: um Caminhão IS-A Veículo, ou seja, um caminhão é um tipo mais específico de veículo. Toda classe que não declara **extends** explicitamente herda, de forma implícita, de `java.lang.Object` — a raiz de qualquer hierarquia em Java.

Um detalhe que costuma passar despercebido: campos `private` da superclasse existem no objeto, mas não ficam visíveis diretamente para a subclasse. Já campos `protected` são herdados e acessíveis nas subclasses — o que reforça, mais uma vez, por que getters e setters são importantes: eles dão acesso controlado independentemente do nível de visibilidade do campo.

```
public class Veiculo {
    protected String placa;
    public void abastecer() {
        System.out.println(placa + " foi abastecido.");
    }
}

public class Motocicleta extends Veiculo {
    private boolean temBau;
    // abastecer() é herdado de Veiculo, sem duplicar código
    public void ligarPisca() { System.out.println("Pisca ligado."); }
}

// Uso:
Motocicleta moto = new Motocicleta();
moto.placa = "MTC-9921"; // protected — acessível na subclasse
moto.abastecer(); // herdado de Veiculo
moto.ligarPisca(); // exclusivo de Motocicleta
```

REGRA IS-A

Antes de usar herança, verifique se a relação faz sentido no mundo real: Motocicleta IS-A Veículo funciona; forçar uma relação que não é verdadeiramente "é um tipo de" costuma gerar hierarquias confusas mais adiante.

Override: redefinindo o comportamento do pai

Fazer **override** (sobrescrever) significa declarar, na subclasse, um método com exatamente a mesma assinatura — nome e parâmetros — de um método já existente na superclasse. A partir daí, chamar esse

método em um objeto da subclasse executa a versão da subclasse, não a do pai. A anotação **@Override** não é obrigatória, mas vale sempre usá-la: ela faz o compilador conferir se realmente existe um método correspondente na superclasse, evitando o erro silencioso de criar sem querer um método novo em vez de sobrescrever o existente. Métodos **static**, **private** e **final** não podem ser sobrescritos.

```
public class Veiculo {
    protected String modelo;
    @Override
    public String toString() { return "Veiculo generico: " + modelo; }
}

public class Caminhao extends Veiculo {
    private double capacidadeToneladas;
    @Override
    public String toString() {
        return super.toString() + " | Caminhao, capacidade: " + capacidadeToneladas + "t";
    }
}

// Polimorfismo em ação:
Veiculo v = new Caminhao(); // variável do tipo Veiculo, objeto real Caminhao
System.out.println(v); // executa Caminhao.toString(), não Veiculo.toString()
```


static, POJO e a classe Object

Membros de classe, objetos simples de dados e a raiz de toda hierarquia

Campos e métodos static

Uma variável de instância existe uma vez por objeto — cada instância guarda sua própria cópia. Uma variável **static**, ao contrário, pertence à classe como um todo: existe uma única cópia compartilhada entre todos os objetos criados a partir dela. Métodos static seguem a mesma lógica — pertencem à classe, não têm acesso a `this` nem a campos de instância, e só podem manipular membros também static. Um método de instância, por sua vez, pode acessar tanto campos de instância quanto campos static. Regra prática: se um método não depende de nenhum dado específico do objeto, ele é candidato natural a ser static.

```
public class Frota {
    private static int totalDeVeiculos = 0; // uma única cópia para toda a frota
    private int codigoInterno; // cópia própria de cada veículo

    public Frota() {
        totalDeVeiculos++;
        this.codigoInterno = totalDeVeiculos;
    }

    public static int getTotalDeVeiculos() { return totalDeVeiculos; } // static
    public int getCodigoInterno() { return codigoInterno; } // de instância
}

// Uso:
Frota v1 = new Frota(); // totalDeVeiculos=1
Frota v2 = new Frota(); // totalDeVeiculos=2
System.out.println(Frota.getTotalDeVeiculos()); // 2 — chamado pela classe
```

POJO — Plain Old Java Object

Um POJO é uma classe simples, cujo único papel é carregar dados — sem lógica de negócio complexa e sem depender de nenhum framework. Também é chamado de JavaBean, Entity ou DTO (Data Transfer Object), e é a forma mais comum de representar informação que trafega entre partes diferentes de uma aplicação — por exemplo, os dados de manutenção de um veículo vindos de um formulário. Um POJO típico tem campos private, um ou mais construtores, getters e setters públicos, e normalmente um `toString()` sobrescrito para facilitar a depuração.

```

public class RegistroDeManutencao {
    private String placaVeiculo;
    private String tipoServico;
    private double custo;

    public RegistroDeManutencao(String placaVeiculo, String tipoServico, double custo) {
        this.placaVeiculo = placaVeiculo;
        this.tipoServico = tipoServico;
        this.custo = custo;
    }

    public String getPlacaVeiculo() { return placaVeiculo; }
    public String getTipoServico() { return tipoServico; }
    public double getCusto() { return custo; }

    @Override
    public String toString() {
        return placaVeiculo + " - " + tipoServico + " - R$ " + custo;
    }
}

```

java.lang.Object — a raiz de tudo

Toda classe em Java herda implicitamente de `java.lang.Object`, mesmo quando `extends` não é escrito em lugar nenhum. Isso garante que qualquer objeto criado tenha, de fábrica, métodos como `toString()`, `equals(Object)` e `hashCode()`, todos herdados de `Object`. A versão padrão de `toString()` devolve algo pouco legível, no formato `NomeDaClasse@endereco` — por isso ela é quase sempre sobrescrita para retornar uma descrição útil do objeto.

```

public class Veiculo { // implicitamente extends Object
    private String placa;

    @Override
    public boolean equals(Object obj) {
        if (this == obj) return true;
        if (!(obj instanceof Veiculo)) return false;
        return this.placa.equals(((Veiculo) obj).placa);
    }
}

```

Overloading vs Overriding

Dois conceitos que parecem parecidos, mas resolvem problemas diferentes

Duas ideias que costumam ser confundidas

Overloading e overriding têm um ponto em comum — os dois envolvem métodos com o mesmo nome — mas funcionam de maneiras completamente diferentes e resolvem problemas diferentes. Entender a distinção evita um dos erros clássicos de quem está aprendendo orientação a objetos.

Conceito	O que significa
Nome do método	Igual nos dois casos.
Parâmetros	Overloading: diferentes. Overriding: idênticos (mesma assinatura).
Onde ocorre	Overloading: na mesma classe. Overriding: entre subclasse e superclasse.
Tipo de retorno	Overloading: pode variar livremente. Overriding: igual ou covariante.
Modificador de acesso	Overloading: qualquer um. Overriding: não pode ficar mais restritivo.
Quando é decidido	Overloading: em tempo de compilação. Overriding: em tempo de execução.
Também chamado de	Overloading: polimorfismo estático. Overriding: polimorfismo dinâmico.

Covariant return type e regras de acesso

Para que um método conte como override, ele precisa manter o mesmo nome e os mesmos parâmetros da versão do pai, e o tipo de retorno precisa ser igual ou **covariante** — ou seja, pode ser um subtipo do que o método original retornava. O modificador de acesso também segue uma regra: não pode ficar mais restritivo do que estava na superclasse. Se o pai declarou o método como `protected`, o filho pode mantê-lo `protected` ou abri-lo para `public`, mas nunca reduzi-lo a `private`.

```
// Covariant return type:
class Veiculo {
    public Veiculo criarUnidadeGemea() { return new Veiculo(); }
}
class Caminhao extends Veiculo {
    @Override
    public Caminhao criarUnidadeGemea() { return new Caminhao(); } // Caminhao IS-A Veiculo,
    ok!
}

// Regra de modificador de acesso:
class VeiculoBase {
    protected void revisar() { }
}
class VeiculoPremium extends VeiculoBase {
    @Override
    public void revisar() { } // OK: public é menos restritivo que protected
    // private void revisar() { } // ERRO: private é mais restritivo, não compila
}
```

Resumo da Apostila

Os conceitos essenciais desta apostila, em uma única tabela de consulta

Conceito	O que significa
Classe	O molde que descreve campos e métodos. Não ocupa memória heap sozinha.
Objeto / Instância	Criado com new. Cada um tem sua própria cópia dos campos no heap.
Referência	O que uma variável de objeto realmente guarda: um endereço, não o objeto em si.
null	Ausência de objeto. Chamar um método em null lança NullPointerException.
Encapsulamento	Campos private + getters/setters com validação, controlando o acesso ao estado.
Construtor	Inicializa o objeto. Sem tipo de retorno. Pode ser sobrecarregado.
this() / super()	Encadeiam construtores: this() chama outro construtor da mesma classe; super() chama o do pai.
Herança (extends)	Subclasse herda campos e métodos não-private da superclasse. Relação IS-A.
Override (@Override)	Subclasse redefine um método do pai com a mesma assinatura. Decidido em runtime.
static	Pertence à classe — uma única cópia compartilhada, em vez de uma por objeto.
POJO	Classe simples de dados: campos private, getters/setters, construtor, toString().
java.lang.Object	Superclasse implícita de qualquer classe Java; origem de toString(), equals() e hashCode().
Overloading vs Overriding	Overloading: mesmo nome, parâmetros diferentes, decidido em compilação. Overriding: mesma assinatura, decidido em runtime.